

Build Spec: Natural-Language-Driven Dynamic Section Customization

Context

I have an existing React + TypeScript analytics dashboard (client intelligence platform). Each client has multiple sections — Financial Overview, Company Overview, Competitors, Vendors, Leadership, Employee List, News, White Space Opportunities, Risks, Acquisitions, ESG, etc. Data for every section is generated by calling Gemini with a section-specific prompt.

Current pain: Whenever a business user wants a field added, removed, or changed in a section (e.g., add "Free Cash Flow" to Financial Overview, remove "LinkedIn Profile" from Employee List, add a quarterly breakdown column to a table), a developer has to manually update the Gemini prompt, backend types, frontend interfaces, and React components, then redeploy. This happens constantly and is the single biggest maintenance burden on the project.

Goal: Let an admin user request field-level changes to any section in plain natural language, have the system translate that into an updated Gemini prompt automatically, and have the frontend render whatever shape of data comes back — with zero code changes and zero redeployments for these field-level edits.

Architecture Decisions Already Made (do not relitigate these)

1. **Layout shape stays fixed per section type; only fields inside it are dynamic.** Each section has one of a small set of pre-built layout templates: `kpi-grid`, `table`, `timeline`, `list`, `matrix`. A developer builds each template once. The LLM never generates layout/JSX — only field lists and prompt text change. This was chosen deliberately over full generative-layout or LLM-generated-JSX approaches, which were rejected as too fragile/risky for this use case.
2. **No predefined checkbox list of "allowed" fields.** The user is not choosing from a bounded set defined in code. They see the *currently rendered fields* for a section and can describe changes in natural language (add, remove, rename, restructure).

3. **A helper LLM call — not hardcoded string manipulation — handles turning a natural language request into an updated prompt + updated expected output schema.** This keeps the system unbounded without needing to hardcode every possible field a user might ask for.
4. **Human approval gate before any change goes live.** The helper LLM proposes an updated prompt + schema; the admin reviews and approves before it replaces the production prompt for that section.
5. **Two LLM roles, potentially two different models** (user has API access to multiple Gemini models via a subscription/API key — pick appropriate ones):
 - **Prompt/Schema Refiner** — takes the current prompt, the last known output shape, and the user's natural language request; returns a revised prompt plus a revised lightweight schema descriptor. This is a low-latency, structured-output task — favor a faster/cheaper model with strong instruction-following and reliable JSON mode.
 - **Production Data Extractor** — the existing Gemini call that actually pulls the section's data using the approved prompt. Keep using whatever model is currently used for data quality; no change needed here except it now

reads its prompt from the DB instead of a hardcoded string.

6. **Zod validation stays as a safety net**, not as a bounding mechanism. It validates that whatever Gemini returns is at least well-formed enough to render (correct types, no missing required wrapper keys) – it does not restrict which field names are allowed.

System Flow

```
[Admin clicks "Edit" on a section]
  |
  v
[UI shows current fields being rendered + a
natural language input box]
  |
  v
[Admin types request, e.g. "Add quarterly
revenue split by ecommerce vs services,
remove LinkedIn column"]
  |
  v
[Backend: Prompt/Schema Refiner LLM call]
  input: { currentPrompt, currentSchema
(last known output shape), userRequest }
  output: { revisedPrompt, revisedSchema,
changeSummary }
  |
```

```

      v
[UI shows a diff/preview: "Here's what will
change" – old fields vs new fields, plain-
English summary]
      |
      v
[Admin clicks Approve]
      |
      v
[Backend stores revisedPrompt +
revisedSchema as the new active version for
that section]
      (versioned – keep history, allow
rollback)
      |
      v
[Production Extractor LLM call runs with
the new prompt]
      |
      v
[Zod-validated response stored in DB]
      |
      v
[Frontend fetches section data + schema
descriptor, renders via the generic dynamic
renderer matching the section's fixed
layout template]
```

Data Model

Design tables/collections for:

section_configs

- `sectionId`, `clientId`, `sectionType` (enum: financial-overview, employee-list, etc.)
- `layoutTemplate` (enum: kpi-grid | table | timeline | list | matrix)
- `activePromptVersionId`
- `timestamps`

prompt_versions (full history, never overwritten)

- `id`, `sectionConfigId`, `promptText`, `schemaDescriptor` (JSON), `changeSummary`, `createdBy`, `approvedBy`, `approvedAt`, `status` (draft | approved | active | superseded)

section_data

- `sectionConfigId`, `promptVersionId` (which version produced this data), `data` (JSON, matches `schemaDescriptor`), `fetchAt`, `validationStatus`

schemaDescriptor shape (this is what both LLM calls and the frontend renderer share — keep it intentionally lightweight, NOT a full JSON Schema):

```
{
  "layoutTemplate": "table",
  "fields": [
    { "key": "employeeName", "label":
      "Name", "type": "text" },
```

```
{ "key": "designation", "label":  
  "Designation", "type": "text" },  
  { "key": "email", "label": "Email",  
    "type": "text" },  
  { "key": "hireDate", "label": "Hire  
Date", "type": "date" }  
]  
}
```

Types to support: text, number, currency,
percentage, date, url, badge, boolean.

Backend Endpoints Needed

- `POST /sections/:id/refine` — takes user's natural language request, calls the Refiner LLM, returns `{revisedPrompt, revisedSchema, changeSummary}` as a **draft**, does not persist as active yet.
- `POST /sections/:id/approve` — takes a draft's id, marks it approved+active, triggers the production extractor run.
- `GET /sections/:id/data` — returns current data + `schemaDescriptor` for rendering.
- `GET /sections/:id/history` — version history for rollback/audit.
- `POST /sections/:id/rollback/:versionId` — reactivate a prior prompt version.

Frontend Requirements

1. **Generic dynamic renderer** — one component per `layoutTemplate` (`KpiGrid`, `DataTable`, `Timeline`, `ListView`, `Matrix`). Each accepts `{schemaDescriptor, data}` as props and renders purely from that — no hardcoded field names anywhere in these components.
2. **Type-based cell/field formatting** — a shared formatter util that takes `{value, type}` and returns the right display (currency formatting, date formatting, badge coloring, link rendering, etc.), used by all layout components.
3. **Edit mode UI per section** — small pencil/edit icon in the section header. Opens a panel showing:
 - Current fields (read-only chips/list, pulled from the active `schemaDescriptor`)
 - A text input: "Describe what you want to change"
 - Submit → calls `/refine` → shows a preview diff (added fields, removed fields, changed prompt summary) → Approve/Discard buttons
4. **Zod schemas** for `schemaDescriptor` and for each layout template's expected `data` shape, used to validate before render; on validation

failure, show a graceful fallback ("This section couldn't be displayed — contact an admin") instead of crashing.

Folder Structure

```
src/  
  renderer/  
    layouts/  
      KpiGrid.tsx  
      DataTable.tsx  
      Timeline.tsx  
      ListView.tsx  
      Matrix.tsx  
      LayoutResolver.tsx      // picks  
layout component by  
schemaDescriptor.layoutTemplate  
  fieldFormatters.ts  
  sections/  
    SectionContainer.tsx      // fetches  
data+schema, renders via LayoutResolver,  
hosts edit button  
    EditSectionPanel.tsx      // natural  
language edit UI + diff preview  
  schema/  
    schemaDescriptor.types.ts  
    schemaDescriptor.zod.ts  
  api/  
    sectionsApi.ts  
  types/  
    section.types.ts
```

```
backend/  
  llm/  
    promptRefiner.ts      // calls the  
Refiner model, structured JSON output  
    dataExtractor.ts      // existing  
Gemini call, now reads prompt from DB  
  routes/  
    sections.ts  
  db/  
    models/ (sectionConfig, promptVersion,  
sectionData)
```

Deliverables Expected From This Build

1. TypeScript types for `SchemaDescriptor`, `Field`, `LayoutTemplate`
2. Zod schemas for descriptor + each layout's data shape
3. `LayoutResolver` + all five layout components, fully generic, driven only by props
4. `fieldFormatters.ts` covering all listed types
5. Backend `promptRefiner.ts` — the LLM call that turns `{currentPrompt, currentSchema, userRequest}` into `{revisedPrompt, revisedSchema, changeSummary}`, with a strict JSON-mode system prompt so it never returns free text

6. Backend `dataExtractor.ts` refactor to read active prompt from DB
7. All API endpoints listed above
8. DB schema/migration for the three tables
9. `SectionContainer.tsx` + `EditSectionPanel.tsx` with the approve/diff-preview flow
10. One fully working end-to-end example using **Employee List** (table layout) showing: current 3 fields → user requests "add email, remove LinkedIn" → refiner proposes change → approve → new data fetched → table now shows 3 different columns, with zero edits to `DataTable.tsx`

Build this section by section, starting with the shared renderer + types, then the backend refiner flow, then wiring the Employee List example end to end.